CS 240 - Lab 6

TAs: Aditya & Rishi

Spring 2025

Welcome to the sixth lab of this course!!

Instructions:

- This lab has 4 questions. All questions are completely ungraded!!

Q1. SMO for Training SVMs

In the previous sections, we studied classifiers derived from probabilistic modeling assumptions. Support Vector Machines (SVMs) adopt a different principle: instead of modeling class probabilities, they construct a decision boundary that maximizes the geometric margin between classes. Remarkably, the optimal boundary depends only on a subset of training samples, called the **support vectors**. These are the points closest to the separating hyperplane; removing any other point does not change the classifier.

To simplify the presentation, we absorb the bias term into the feature representation and work with a homogeneous decision function. Consequently, the classifier is expressed purely through inner products, and no explicit intercept parameter appears.

Dual Optimization Problem. Given training data $\{(x_i, y_i)\}_{i=1}^n$ with labels $y_i \in \{-1, +1\}$, the dual SVM objective is

$$\max_{\alpha} W(\alpha) \quad \text{where} \quad W(\alpha) = \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} y_i y_j \alpha_i \alpha_j K(x_i, x_j),$$

subject to

$$0 \le \alpha_i \le C, \qquad \sum_{i=1}^{n} y_i \alpha_i = 0.$$

Here $C > 0$ controls the margin–error tradeoff and $K(x_i, x_j)$ is a kernel function.

After optimization, the classifier is

$$f(x) = \sum_{i=1}^{n} y_i \alpha_i K(x_i, x).$$

Only points with $\alpha_i > 0$ influence the decision boundary; these are the support vectors.

Kernel Function. A kernel computes inner products in an implicit feature space,

$$K(x_i, x_j) = \langle \phi(x_i), \phi(x_j) \rangle.$$

In this lab we use the radial basis function (RBF) kernel

$$K(x_i, x_j) = \exp(-\gamma \|x_i - x_j\|^2),$$

which enables nonlinear decision boundaries.

Sequential Minimal Optimization

Solving the dual problem directly requires quadratic programming. Sequential Minimal Optimization (SMO) instead optimizes two multipliers at a time while keeping the rest fixed. This is possible because the equality constraint

$$\sum_i y_i \alpha_i = 0$$

forces any change in one multiplier to be balanced by another.

Each step of SMO optimizes two Lagrange multipliers. Without loss of generality, let these multipliers be α_1 and α_2. The dual objective restricted to these variables can be written as

$$W(\alpha_1, \alpha_2) = \alpha_1 + \alpha_2 - \frac{1}{2}K_{11}\alpha_1^2 - \frac{1}{2}K_{22}\alpha_2^2 - sK_{12}\alpha_1\alpha_2 - y_1\alpha_1 v_1 - y_2\alpha_2 v_2 + W_{\text{constant}}$$

where

$$K_{ij} = K(x_i, x_j) \quad \text{and} \quad v_i = \sum_{j \neq 1,2} y_j \alpha_j^{\text{old}} K_{ij} = f^{\text{old}}(x_i) - y_1\alpha_1^{\text{old}}K_{1i} - y_2\alpha_2^{\text{old}}K_{2i}$$

Note that variables with superscript "old" denote values from the previous iteration. W_{constant} collects terms independent of α_1, α_2.

The two variables must satisfy the linear equality constraint

$$\alpha_1 + s\alpha_2 = \alpha_1^{\text{old}} + s\alpha_2^{\text{old}} \equiv \gamma \quad \text{where} \quad s = y_1 y_2$$

Solving for α_1,

$$\alpha_1 = \gamma - s\alpha_2.$$

Substituting into W gives the objective along the feasible line:

$$\begin{aligned} W(\alpha_2) = (\gamma - s\alpha_2) + \alpha_2 - \frac{1}{2}K_{11}(\gamma - s\alpha_2)^2 - \frac{1}{2}K_{22}\alpha_2^2 \\ - sK_{12}(\gamma - s\alpha_2)\alpha_2 - y_1(\gamma - s\alpha_2)v_1 - y_2\alpha_2 v_2 + W_{\text{constant}} \end{aligned}$$

Differentiating with respect to α_2,

$$\frac{dW}{d\alpha_2} = sK_{11}(\gamma - s\alpha_2) - K_{22}\alpha_2 + K_{12}\alpha_2 - sK_{12}(\gamma - s\alpha_2) + y_2 v_1 - s - y_2 v_2 + 1$$

The second derivative along the constraint is

$$\frac{d^2W}{d\alpha_2^2} = -(K_{11} + K_{22} - 2K_{12}) \equiv -\eta.$$

If $\eta > 0$, the maximum occurs where the first derivative vanishes. Setting $\frac{dW}{d\alpha_2}$ as 0 gives

$$\alpha_2^{\text{new}}(K_{11}+K_{22}-2K_{12}) = s(K_{11}-K_{12})\gamma + y_2(v_1-v_2) + 1 - s$$

Expanding out the value of γ and v yields,

$$\alpha_2^{\text{new}} = \alpha_2^{\text{old}} + \frac{y_2(f^{\text{old}}(x_1) - f^{\text{old}}(x_2) + y_2 - y_1)}{\eta}, \qquad \eta = K_{11}+K_{22}-2K_{12}.$$

Using the definitions of the errors $E_i = f^{\text{old}}(x_i) - y_i$, one obtains

$$\boxed{\alpha_2^{\text{new}} = \alpha_2^{\text{old}} + \frac{y_2(E_1 - E_2)}{\eta}}.$$

Feasibility bounds. The updated multipliers must satisfy the box constraints $0 \le \alpha_i \le C$ and the equality constraint $\alpha_1 + s\alpha_2 = \gamma$.

If $y_1 \neq y_2$ ($s = -1$), the bounds on α_2 are

$$L = \max\Big(0,\ \alpha_2^{\text{old}} - \alpha_1^{\text{old}}\Big), \qquad H = \min\Big(C,\ C + \alpha_2^{\text{old}} - \alpha_1^{\text{old}}\Big).$$

If $y_1 = y_2$ ($s = 1$), the bounds become

$$L = \max\Big(0,\ \alpha_1^{\text{old}} + \alpha_2^{\text{old}} - C\Big), \qquad H = \min\Big(C,\ \alpha_1^{\text{old}} + \alpha_2^{\text{old}}\Big).$$

$$\alpha_2^{\text{new, clipped}} = \begin{cases} H, & \alpha_2^{\text{new}} \ge H, \\ \alpha_2^{\text{new}}, & L < \alpha_2^{\text{new}} < H, \\ L, & \alpha_2^{\text{new}} \le L. \end{cases}$$

The updated α_1 is then recovered from the equality constraint:

$$\alpha_1^{\text{new}} = \alpha_1^{\text{old}} + s\left(\alpha_2^{\text{old}} - \alpha_2^{\text{new, clipped}}\right).$$

Selecting multipliers (Platt's SMO heuristic).

Note: While we will simply iterate over the training examples to find violations, the outer examples can often be chosen by finding examples whose E_i term is the largest, plugging this into the gradient, we observe that this violates the KKT conditions the most.

The efficiency of SMO depends critically on selecting multipliers that lead to meaningful improvement. At optimality, the Karush–Kuhn–Tucker (KKT) conditions require

$$\alpha_i = 0 \Rightarrow y_i f(x_i) \ge 1, \quad 0 < \alpha_i < C \Rightarrow y_i f(x_i) = 1, \quad \alpha_i = C \Rightarrow y_i f(x_i) \le 1.$$

During training, many multipliers violate these conditions. SMO iteratively selects such violators and updates them.

Step 1: Choosing the first multiplier.

Compute the prediction error

$$E_i = f(x_i) - y_i.$$

Note that $y_i E_i$ can also be written as $y_i f(x_i) - 1$ since $y_i y_i = 1$ when $y_i \in \{+1, -1\}$ An index i is eligible for update if it violates the KKT conditions:

$$y_i E_i < -\tau \ \text{ and } \ \alpha_i < C, \qquad \text{or} \qquad y_i E_i > \tau \ \text{ and } \ \alpha_i > 0,$$

where τ is a small tolerance.

Step 2: Choosing the second multiplier. After selecting i, choose a distinct index j to maximize the magnitude of the update. From the analytical update rule, the step size is proportional to $|E_i - E_j|$, so SMO selects

$$j = \arg\max_k |E_i - E_k|.$$

This choice typically produces the largest improvement in the objective.

Preference for non-bound multipliers. Multipliers satisfying $0 < \alpha_k < C$ (called *non-bound* multipliers) lie on the margin and provide the most informative updates. The algorithm first searches this set when choosing j. If no progress is made, it expands the search to all training points.

Fallback strategy. If no suitable j is found or the step is numerically insignificant, the algorithm tries:

- other non-bound multipliers,
- the entire training set,
- a random index $j \neq i$ as a last resort.

Maintaining prediction errors (error cache)
During training, the algorithm frequently evaluates

$$E_i = f(x_i) - y_i,$$

which measures how far the current model prediction is from satisfying the margin condition. Recomputing $f(x_i)$ from scratch requires summing over all training points and is computationally expensive.

To improve efficiency, maintain an *error cache* storing the current values of E_i for all training points. After updating a pair of multipliers (α_i, α_j), the predictions change only due to these updates. Therefore, the errors can be updated incrementally rather than recomputed:

$$E_k \leftarrow E_k + y_i \Delta\alpha_i K(x_i, x_k) + y_j \Delta\alpha_j K(x_j, x_k),$$

where $\Delta\alpha_i$ and $\Delta\alpha_j$ are the changes in the multipliers.

After selecting (i, j), the pair is updated using the analytical steps derived earlier. The process repeats until no multipliers violate the KKT conditions.

Algorithm 1 Sequential Minimal Optimization (SMO)

Initialize $\alpha_i \leftarrow 0$ for all i
Compute kernel matrix $K(x_i, x_j)$
Compute predictions $f(x_i)$ and errors $E_i = f(x_i) - y_i$
repeat
    numChanged $\leftarrow 0$
    for each training example i **do**
        if α_i violates KKT conditions **then**
            Select $j \neq i$ maximizing $|E_i - E_j|$ (prefer non-bound multipliers)
            Compute bounds L, H and negative curvature η
            if $L = H$ or $\eta \leq 0$ **then**
                continue
            end if
            Update α_j using analytical rule and clip to $[L, H]$
            Update α_i to maintain constraint
            Update $f(x)$ and error cache
            numChanged $\leftarrow$ numChanged $+ 1$
        end if
    end for
until numChanged $= 0$ (no KKT violations remain)

Q2. Solving the SVM Dual using Quadratic Programming

In the previous question, you implemented Sequential Minimal Optimization (SMO) to solve the dual optimization problem of the Support Vector Machine. In this question, you will solve the **same dual problem** using a general-purpose quadratic programming (QP) solver.

This exercise emphasizes that training an SVM is a convex optimization problem and provides an alternative method for computing the optimal multipliers.

Dual Optimization Problem.

Given training data $\{(x_i, y_i)\}_{i=1}^n$ with labels $y_i \in \{-1, +1\}$, the dual SVM objective is

$$\max_{\alpha} \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} y_i y_j \alpha_i \alpha_j K(x_i, x_j)$$

subject to

$$0 \leq \alpha_i \leq C, \qquad \sum_{i=1}^{n} y_i \alpha_i = 0.$$

After optimization, the decision function is

$$f(x) = \sum_{i=1}^{n} y_i \alpha_i K(x_i, x).$$

Only points with $\alpha_i > 0$ influence the classifier; these are called the **support vectors**.

Quadratic Programming Form. A standard quadratic program minimizes an objective of the form

$$\min_{\alpha} \frac{1}{2} \alpha^T P \alpha + q^T \alpha$$

subject to linear constraints of the form

$$G\alpha \leq h, \qquad A\alpha = b,$$

where the inequality constraints define a feasible region and the equality constraints impose linear relationships among the variables.

Show that the SVM dual problem can be written in this standard QP form by appropriately defining the matrices and vectors involved.

Tasks

1. Derive the quadratic programming formulation corresponding to the SVM dual objective.
2. Using a quadratic programming solver `cvxopt`, compute the optimal multipliers α.

Q3. Support Vector Regression (SVR)

The goal of this assignment is to implement and analyze ε-Support Vector Regression (SVR). You will train SVR models using both a linear kernel and a Radial Basis Function (RBF) kernel, and study how the hyperparameters ε and C affect the learned regression function.

Problem Formulation Given training data $\{(x_i, y_i)\}_{i=1}^N$, ε-SVR learns a function that is as flat as possible while allowing prediction errors within an ε margin. The primal optimization problem is

$$\min_{w,b,\xi_i^+,\xi_i^-} \frac{1}{2}\|w\|^2 + C\sum_{i=1}^{N}(\xi_i^+ + \xi_i^-)$$

subject to

$$y_i - (w^\top x_i + b) \le \varepsilon + \xi_i^+, \qquad (w^\top x_i + b) - y_i \le \varepsilon + \xi_i^-, \qquad \xi_i^+, \xi_i^- \ge 0$$

Here, ξ_i^+ and ξ_i^- are slack variables that allow violations of the ε-insensitive region, and $C > 0$ controls the trade-off between model flatness and training error.

To enable nonlinear regression, SVR is typically solved in its dual formulation using the kernel trick. Introducing Lagrange multipliers α_i and α_i^* for the inequality constraints leads to the regression function

$$f(x) = \sum_{i=1}^{N}(\alpha_i - \alpha_i^*)K(x_i, x) + b,$$

where $K(\cdot,\cdot)$ is the kernel function. In this assignment you will experiment with both the linear kernel and the RBF kernel.

Tasks:

- **Dataset Generation** Generate synthetic regression data according to

  $$y = \sin(x) + \varepsilon \qquad \varepsilon \sim N(0, 0.1)$$

  Sample 100 points uniformly from the interval $[-3, 3]$, and split it into train and test dataset.
- Implement ϵ-SVR using both a linear kernel and an RBF kernel. Feel free to explore SMO or QP for the same.
- Train the model on the generated dataset and visualize the learned regression function. Your plots should include the true function $\sin(x)$, the training points, the SVR prediction, the ϵ-tube around the prediction function, and the identified support vectors.
- Study the effect of the hyperparameters $\epsilon \in \{0.01, 0.1, 0.5\}$ and $C \in \{0.1, 1, 100\}$. For the RBF kernel, also experiment with different values of the kernel parameter γ.

Analysis Questions

Answer the following conceptual questions:

1. Why does increasing ϵ typically reduce the number of support vectors?
2. What happens to the learned function when C becomes very large?
3. Why is the SVR solution sparse?
4. How does SVR compare to Ridge Regression in terms of objective function and behavior?

Q4. Learning a Weighted RBF Kernel via Gradient Descent

The goal of this assignment is to understand kernel methods in their dual formulation and to implement Kernel Ridge Regression (KRR). In addition, you will learn how kernel parameters can themselves be optimized using gradient-based methods. In particular, you will implement a parameterized kernel and learn its parameters by minimizing validation loss. This exercise will require deriving gradients and implementing gradient descent for kernel parameter learning.

Kernel Ridge Regression

Kernel Ridge Regression (KRR) is a kernelized version of ridge regression that enables nonlinear regression through the kernel trick. Suppose we are given training data

$$\{(x_i, y_i)\}_{i=1}^n, \quad x_i \in \mathbb{R}^d,\ y_i \in \mathbb{R}.$$

Let $\phi(x)$ denote a feature map into a possibly high-dimensional Hilbert space. Ridge regression in the feature space solves the optimization problem

$$\min_w \frac{1}{2} \sum_{i=1}^n \Big(y_i - w^\top \phi(x_i)\Big)^2 + \frac{\lambda}{2}\|w\|^2$$

By the Representer Theorem, the optimal solution lies in the span of the training features, and can therefore be written as

$$w = \sum_{i=1}^n \alpha_i \phi(x_i).$$

Substituting this representation into the objective yields the dual solution

$$\alpha = (K + \lambda I)^{-1} y,$$

where the matrix K is the kernel (Gram) matrix defined by

$$K_{ij} = K(x_i, x_j) = \langle \phi(x_i), \phi(x_j) \rangle.$$

Predictions for a new point x are then given by

$$f(x) = \sum_{i=1}^n \alpha_i K(x_i, x).$$

Weighted RBF Kernel

In this assignment, we use a parameterized kernel known as the weighted RBF kernel:

$$K(x, z; \mathbf{w}) = \exp\left(-\sum_{k=1}^{d} w_k (x_k - z_k)^2\right),$$

where $\mathbf{w} = (w_1, \ldots, w_d)$ and each $w_k > 0$. This kernel allows different feature dimensions to have different levels of importance, since the parameter w_k controls how strongly differences in the k-th feature affect similarity.

Kernel Ridge Regression with Weighted RBF

Given training data $\{(x_i, y_i)\}_{i=1}^n$, we first construct the kernel matrix

$$K_{ij}(\mathbf{w}) = \exp\left(-\sum_{k=1}^{d} w_k (x_{ik} - x_{jk})^2\right).$$

The Kernel Ridge Regression solution is obtained by solving

$$A\alpha = y, \quad \text{where} \quad A = K(\mathbf{w}) + \lambda I.$$

Note that α should be obtained by solving this linear system rather than explicitly computing a matrix inverse. Predictions for a new input x are then given by

$$f(x) = \sum_{i=1}^{n} \alpha_i K(x_i, x; \mathbf{w}).$$

The above prediction form follows from the **Representer Theorem**. Consider a feature map $\phi(x)$ such that the kernel satisfies

$$K(x_i, x_j) = \langle \phi(x_i), \phi(x_j) \rangle.$$

Kernel ridge regression can then be written as ridge regression in the feature space:

$$\min_w \sum_{i=1}^{n} (y_i - w^\top \phi(x_i))^2 + \lambda \|w\|^2.$$

A basic linear algebra argument shows that the optimal solution $w^\star$ must lie in the span of the training feature vectors $\{\phi(x_i)\}_{i=1}^n$, so it can be written as

$$w^\star = \sum_{i=1}^{n} \alpha_i \phi(x_i).$$

Substituting this into the predictor $f(x) = w^{\star\top} \phi(x)$ gives

$$f(x) = \sum_{i=1}^{n} \alpha_i \langle \phi(x_i), \phi(x) \rangle = \sum_{i=1}^{n} \alpha_i K(x_i, x),$$

which yields the kernel expansion used in kernel ridge regression.

For a proof using basic linear algebra and feature mappings, see: `https://en.wikipedia.org/wiki/Representer_theorem`.

Validation Loss

Although α minimizes the training objective for a fixed kernel parameter vector $\mathbf{w}$, our ultimate goal is to choose $\mathbf{w}$ so that the model generalizes well. To measure this, we evaluate performance on a separate validation dataset.

Let $K_{val}(\mathbf{w})$ denote the kernel matrix between validation points and training points. The prediction error on the validation set is

$$e = K_{val}(\mathbf{w})\alpha(\mathbf{w}) - y_{val}.$$

The validation loss is defined as

$$L(\mathbf{w}) = \frac{1}{2}\|e\|^2.$$

Because the coefficients α have a closed-form expression depending on $\mathbf{w}$, we can treat $\alpha(\mathbf{w})$ as a function of $\mathbf{w}$ and optimize the validation loss directly using gradient descent:

$$w_k \leftarrow w_k - \eta \frac{\partial L}{\partial w_k}.$$

Thus, the problem becomes a two-level optimization:

- Inner level: solve for $\alpha(\mathbf{w})$ using the closed-form solution (training minimization).
- Outer level: update $\mathbf{w}$ to minimize validation loss (hyperparameter learning).

This exercise demonstrates how closed-form solutions can be embedded inside gradient-based optimization, leading to a principled method for learning kernel parameters.

Mathematical Derivation of Gradient

NOTE: K_{val} and K are not the same.

Step 1: Find derivative of K and K_{val}

Find:

$$\frac{\partial K(\mathbf{w})}{\partial w_k}$$

and:

$$\frac{\partial K_{val}(\mathbf{w})}{\partial w_k}$$

Step 2: Find derivative of α

Find:

$$\frac{\partial \alpha(\mathbf{w})}{\partial w_k}$$

where:

$$\alpha = (K(\mathbf{w}) + \lambda I)^{-1} y$$

Step 3: Final Gradient Expression

Using product rule:

$$\frac{\partial L}{\partial w_k} = e^T \left(\frac{\partial K_{val}}{\partial w_k} \alpha + K_{val} \frac{\partial \alpha}{\partial w_k} \right).$$

This is the gradient you must implement.

Positivity constraint on w_k

For the Gram matrix to be semi-positive definite, we want all the w_k to be positive. While there are multiple solutions to overcome this problem, for the sake of this assignment we are going to set $w_k \leftarrow max(0, w_k)$ after each gradient descent step.

Some other ways would be to parametrise $\mathbf{w}$ with θ such that $w_k = e^{\theta_k}$ or using softplus, but you are not required to look into these for this assignment.

Tasks

1. Implement the weighted RBF kernel.
2. Construct:
   - $K_{train}(\mathbf{w})$
   - $K_{val}(\mathbf{w})$
3. Solve Kernel Ridge Regression to compute $\alpha(\mathbf{w})$ (Don't use matrix inversion to find $\alpha(\mathbf{w})$).
4. Derive and implement $\frac{\partial L}{\partial \theta_k}$.
5. Ensure $w_k > 0$.
6. Perform gradient descent on $\mathbf{w}$.
   (The best stopping criteria would be to stop when $||w_{new} - w_{old}||_2^2 \leq \epsilon$, where ϵ is of your choice)

Optional Extensions

- Extend to full Mahalanobis matrix $M \succeq 0$.
- Try different methods of ensuring positivity of $\mathbf{w}$.
- Compare with grid search.